

ECLIPSE: Verifying Memory Safety of CLI Programs for Embedded Linux Systems

Ajay Gandecha
UNC-Chapel Hill

Ben Chesser
UNC-Chapel Hill

Will Zahrt
UNC-Chapel Hill

Abstract—Embedded systems can be found everywhere in our 21st century lives. The consequences of an attacker exploiting memory-safety vulnerabilities in software running on these systems are catastrophic, allowing attackers to read sensors or interact with the real world through these computers. To address this problem, we introduce a tool, ECLIPSE, for formally verifying C-based command-line utility programs running on embedded Linux-based systems, like the Raspberry Pi, for memory safety violations. ECLIPSE will rely on symbolic execution to systematically explore all feasible program paths and generate concrete inputs to the program that trigger memory safety violations. To help limit path explosion, our tool will utilize an SMT solver that can identify and prune infeasible paths early, and pre-processing that adds bounds on loop iterations and inputs as needed. In order to prove success, we plan to create utility programs that intentionally include memory safety violations and run our tool on it to show exploits. As a stretch goal, our tool may also include an automatic exploit generator, which uses the results of symbolic execution to generate exploits that automatically trigger memory violations in the input C command line program.

I. INTRODUCTION

Though often overlooked, embedded systems support important infrastructure in our everyday lives. Fields including electronics, healthcare, emergency systems, home appliances, locomotion, and automation all rely on these embedded systems functioning safely and as designed.

Embedded computers, including the Raspberry Pi, include general-purpose input / output (GPIO) pins, which serve as the interface between the processor and external hardware components. These pins are software-controlled. Through GPIO pins, software can read both inputs (for example, from sensors or switches) and send outputs (for example, to power lights or motors). If an attacker could exploit software programs running on such embedded systems, they could gain unauthorized access to the hardware itself, including these GPIO pins. This could be especially catastrophic, since the attacker could potentially gain access to observe or interact with the physical world via the affected computer.

Prior work on symbolic execution for embedded systems has largely focused on real firmware targeting specific microcontrollers [6] or full firmware image analysis frameworks requiring hardware emulation [11]. Tools like KLEE have been applied extensively to general-purpose software but not specifically to the class of command-line utilities that run on embedded Linux systems like the Raspberry Pi. The goal of this paper is to formally verify C-based command-line utility

programs running on embedded Linux-based systems, such as the Raspberry Pi, for memory safety violations.

To accomplish this goal, we will introduce ECLIPSE (Embedded CLI Program Symbolic Executor), a tool that finds memory safety vulnerabilities in provided C-based command line utility programs. ECLIPSE relies on symbolic execution to systematically explore all feasible program paths and generate concrete inputs to the program that trigger memory safety violations. Symbolic execution works well with our threat model of a malicious attacker who can control inputs but not modify the program directly [1]. ECLIPSE utilizes KLEE, a leading symbolic execution engine with demonstrated success in detecting memory safety violations in C programs [11] [8].

The primary challenge of symbolic execution is path explosion [2]. To help mitigate this, KLEE leverages an SMT solver to remove infeasible paths during execution. However, this alone is insufficient for CLI-based embedded programs, which often exhibit characteristics that cause the symbolic execution engine to explore a large number of unrealistic, semantically invalid paths, worsening path explosion.

First, CLI programs often accept structured inputs, such as flags and options, whose values are typically constrained (e.g., enumerations or bounded numeric ranges and string lengths). Second, embedded programs often interact with hardware, for example by reading data from GPIO pins, whose values are inherently constrained to a low signal (0) or a high signal (1). However, standard symbolic execution treats both structured inputs and GPIO data as unconstrained, leading to the exploration of semantically invalid paths and contributing to path explosion.

To address this, we introduce a dedicated, automated pre-processor that algorithmically transforms input CLI programs into analysis-friendly representations that limit unnecessary path exploration. The pre-processor analyzes the input program, identifies structured inputs and GPIO reads, and models them as symbolic variables constrained to their valid domains. It additionally generates harness code to enforce these constraints during symbolic execution and applies bounds to loop iterations, which are a common source of path explosion due to unbounded branching [2].

Finally, the pre-processor performs static analysis to identify functions containing code patterns associated with memory-safety risks, such as symbolic array indexing, pointer arithmetic, and calling memory-mutating functions. Using this information, the pre-processor guides symbolic execution toward

these high-risk functions to improve bug-finding efficiency while de-emphasizing the exploration of low-risk paths.

We unify this automated pre-processor with KLEE to create our tool, ECLIPSE, which accepts an input C program and identifies memory safety violations within the program.

To test its effectiveness, we first create mock utility programs that intentionally include memory safety violations. Then, we run ECLIPSE on these programs to ensure that it can correctly identify all the memory safety violations that we introduced. We then run ECLIPSE on larger, existing C-based Linux CLI tools to determine if it can discover any new, previously undiscovered bugs. We acknowledge the possibility that we may correctly implement our entire design for ECLIPSE and still not detect vulnerabilities in open-source programs, especially if they are well-maintained.

We plan to spend the rest of the semester completing this project. We anticipate no additional monetary cost to us, since we already have possession of a Raspberry Pi system if needed for testing purposes.

II. STATE OF THE ART

Foundations of Symbolic Execution

The original papers introducing the symbolic execution method are important background since the goal of our project is to show that this technique is effective in identifying memory safety violations in command-line utilities running on embedded systems [3] [12].

Similarly, a comprehensive survey of symbolic execution techniques gathers and summarizes symbolic execution's strengths, weaknesses, and challenges (like path explosion) through a comprehensive survey [2]. This provides the theoretical framework for the challenges we will face developing ECLIPSE and the solutions we can employ, which include constraint solving via an SMT solver, and constraint reduction.

The KLEE symbolic execution engine is the foundational background for our project, since KLEE is the symbolic execution engine that we plan to integrate into ECLIPSE to verify input programs [1]. KLEE can automatically generate high-coverage test cases by efficiently exploring program paths. In fact, our work is an applied case study of KLEE in the embedded Linux domain.

EXE is a bug-finding tool that creates exploits based on vulnerabilities discovered during symbolic execution [4]. This is relevant to our project because we plan on generating runnable exploits for the command-line programs we are verifying.

Symbolic Execution for Embedded Systems

FIE adapted symbolic execution specifically for microcontroller firmware running on embedded systems [6]. Our work is domain-adjacent because we are extending the application of symbolic execution up the stack from firmware to command-line utilities.

Automated dynamic analysis enables security inspection on embedded systems without physical hardware by emulating the full Linux system environment [5]. Our project is

complementary to this work because, instead of focusing on dynamic analysis through emulation, we are applying symbolic execution to find vulnerabilities.

Avatar combines KLEE's symbolic execution with QEMU's full-system emulation to analyze embedded firmware [11]. Our work is conceptually aligned with this, but instead of focusing on firmware and hardware interaction, we are targeting user-space command line utilities through Linux devices.

Memory Safety Instrumentation and Detection

AddressSanitizer inserts runtime checks around memory addresses, which enables detection of memory errors during program execution [7]. This is important background for our project because we do something similar but instead use symbolic execution to explore execution paths and detect errors before they occur in production.

An analysis of tradeoffs between usability and performance overhead for various compile-time memory safety instrumentations, conducted through the MemInstrument framework, provides context and comparison for our technique, which trades performance for exhaustive path coverage [9].

Hybrid Approaches

Prior approaches to bug detection in embedded environments applies both fuzzing and symbolic execution to detect crashes and vulnerabilities in OpenWRT, which is highly complementary to our project [10]. Instead of casting a wide net over embedded OS distributions, however, we are doing a more targeted project into command-line utilities.

A toolchain combining static analysis and dynamic symbolic execution to detect fault injection vulnerabilities first applies static analysis to identify promising targets and then uses KLEE for targeted symbolic execution [8]. This is directly relevant to our stretch goal of further limiting path explosion through means other than setting bounds.

III. BACKGROUND

Memory safety is a safety property of programs that ensure memory-accesses are well defined and follow the intended rules of the language. Memory-safe programs will never read or write to a memory location in ways that violate language specification or program assumptions. The most common types of memory safety violations include buffer overflows, null pointer de-references, out-of-bounds reads, and memory leaks. These vulnerabilities are particularly prevalent in the C family of programming languages because these languages give direct control to the developer over memory management without built-in safety checks.

Memory safety matters to embedded Linux systems (like the Raspberry Pi) because they rely on small command-line utilities written in C for critical tasks. When a privileged utility contains a memory safety violation, an attacker who controls inputs can potentially do malicious actions such as crashing the system, corrupting data, executing arbitrary code, read sensor inputs or interact with the physical world via GPIO pins, or permanently damaging the hardware.

KLEE is a symbolic execution engine that operates on LLVM bytecode. First, a target C program is compiled to LLVM bytecode using a compiler like Clang. Then, the programmer identifies which inputs should be treated as symbols using KLEE’s intrinsic functions. From there, KLEE executes the bytecode to fork at each symbolic branch and collect path constraints. KLEE performs constraint solving from there by querying the STP constraint solver to check path feasibility and generate concrete values for test cases. Finally, KLEE does error detection and generates files containing concrete input values that allow one to reproduce the path to an error. We plan to use KLEE as the symbolic execution engine that powers ECLIPSE.

IV. THREAT MODEL

The goal of ECLIPSE is to formally verify command line utility programs for embedded Linux devices against memory safety violations. We assume that the attacker can control the inputs to the command line utility, including command line arguments, potential hardware properties including GPIO configurations, and sensor input values. We also assume that the attacker has read-only access to the source code of the program that is the target of attack. The attacker having access to the source code is likely, especially in situations where an attacker attempts to exploit open-source programs.

We also assume that the attacker has no write access to the program’s source code or the program’s binary post-compilation. The goal of our tool is to verify memory safety properties for programs when run with malicious inputs, not tampered code. If the attacker could modify the program directly, they could simply insert malicious behaviour directly into the code rather than needing to exploit a bug to violate a memory safety property.

V. APPROACH

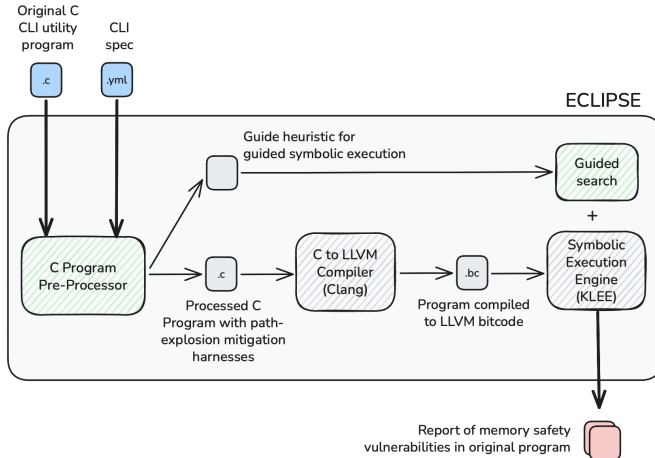


Fig. 1. ECLIPSE’s internal process

At a high level, ECLIPSE operates as a pipeline-style workflow. It accepts two inputs: (1) a C program and (2) a CLI specification file describing the valid input structure.

The program is first transformed by a custom pre-processor to constrain inputs, model hardware interactions, and bound potential sources of path explosion. ECLIPSE then compiles the transformed program into LLVM bytecode using Clang and performs symbolic execution using KLEE with a guided search strategy. Finally, ECLIPSE produces a report describing discovered vulnerabilities along with concrete inputs that reproduce them. An overview of ECLIPSE’s workflow is shown in Figure 1.

ECLIPSE takes in two primary inputs: (1) a C program which is the target CLI utility to analyze, and (2) a YAML-based CLI specification file. The CLI specification file describes the structure of valid input to the target C program and their constraints, such as supported flags and options, data types, and valid values (e.g., enumerations or bounds on integer ranges or string lengths).

For example, consider a program `tool.c` that accepts a `--pin` integer argument that maps to a physical GPIO pin on a Raspberry Pi. These devices have 26 GPIO pins, so `--pin` should only ever accept integer values in the range $\{1, 26\}$. Given this structured argument, the CLI specification file for this program might look like the following:

```

program: tool
arguments:
- name: pin
  flag: --pin
  type: int
  required: true
  min: 0
  max: 26

```

From there, ECLIPSE begins the pre-processing stage by parsing the input C program into an abstract syntax tree (AST). This intermediate representation enables ECLIPSE to algorithmically analyze the program and perform any necessary transformations during the pre-processing stage. Analyzing an AST is more robust than pattern-matching using regex because it analyzes the structure of code directly and is not affected by syntactic anomalies introduced by the developer.

Once the input program is parsed into an AST, the pre-processor examines the program to replace concrete CLI inputs with symbolic variables constrained by the program’s provided CLI specification. Each argument is mapped to a symbolic variable, and constraints are applied based on its declared type. For example, integer arguments may be bound with specified ranges, enumerated arguments may be restricted to a set of possible values, and string inputs may be constrained by length. The pre-processor then generates a harness that enforces these constraints using KLEE’s symbolic execution intrinsic functions. By modeling inputs this way, ECLIPSE ensures that the symbolic execution engine avoids exploring semantically invalid input combinations, which reduces path explosion without sacrificing coverage.

For example, a sample generated harness for the `tool.c` example above, given its CLI specification, might look like the following:

```
int harness_main() {
    int sym_pin;
    char *argv[3];

    // Creates a symbolic variable
    klee_make_symbolic(
        &sym_pin, sizeof(sym_pin), "pin"
    );

    // Constrains the symbolic variable
    klee_assume(sym_pin >= 1 && sym_pin <= 26);

    argv[0] = "tool";
    argv[1] = "--pin";
    argv[2] = int_to_string(sym_pin);

    return main(3, argv);
}
```

With this harness in place, KLEE will never explore paths in the target program where the `pin`'s value is outside its semantically meaningful range of $\{1, 26\}$.

The pre-processor then examines the program to identify GPIO read operations in the AST and replaces them with symbolic variables constrained to the valid domain $\{0, 1\}$. The pre-processor generates a harness that enforces this constraint. Embedded CLI programs frequently interact with hardware through GPIO pins whose inputs are inherently constrained to a low signal (0) or a high signal (1), but by default, the symbolic execution engine would treat these as unconstrained values. Replacing GPIO read operations in the program with constrained symbolic variables ensures that symbolic execution reflects realistic hardware behavior and avoids exploring semantically invalid hardware states, which also helps to reduce path explosion.

Next, the pre-processor analyzes the AST to identify loops and modifies the program to manually add bounds on the maximum number of iterations to limit path explosion. The maximum number of iterations can be customized, but by default, bounds are selected based on the loop's structure (e.g., based on a variable input or fixed with a constant). For example, given an unconstrained loop:

```
while(condition) {
    foo(); // Run some process
}
```

The pre-processor would bound this loop like so:

```
int n = 0;
while(condition && n < MAX_ITERATIONS) {
    foo(); // Run some process
    n += 1;
}
```

This way, no matter when `condition` changes to `false`, the loop always iterates at most `MAX_ITERATIONS` times before terminating.

Finally, the pre-processor performs static analysis by analyzing the AST to identify functions containing code patterns associated with memory-safety risks, such as symbolic array indexing, pointer arithmetic, and calling memory-mutating functions. Functions containing these are then recorded in a *risk file* that ranks functions based on their likelihood of containing memory safety vulnerabilities. This information is later used to guide the symbolic execution engine towards high-risk code with the goal of improving bug-finding efficiency. This risk file, along with the final processed C code file, are output by the pre-processor.

Once pre-processing is finished, ECLIPSE then compiles the processed program. KLEE accepts programs in the format of LLVM bitcode. GCC, the traditional C compiler, compiles C programs into native binary, which is ISA-specific and dependent on the target hardware. Instead of using GCC, ECLIPSE compiles the processed program with Clang, a compiler that compiles C code directly into LLVM bitcode.

Finally, ECLIPSE uses KLEE to perform symbolic execution on the compiled program. KLEE also uses STP, an SMT solver that prunes infeasible paths and generates concrete inputs that satisfy the path constraints generated during symbolic execution. To improve bug-finding efficiency, ECLIPSE augments KLEE's default search strategy with a custom guided search heuristic. This heuristic prioritizes execution paths that lead toward the risky functions noted in the risk file created by the pre-processor. States associated with higher-risk functions are selected more frequently. By default, KLEE interleaves the pre-built `random-path` and `nurs:covnew` heuristics as its search strategy. ECLIPSE replaces the `random-path` with its custom `guided` heuristic. This approach is intentionally lightweight to ensure compatibility with KLEE's existing architecture and so it is reasonably scoped given our time constraint. As a stretch goal, we may try to develop strategies to improve our baseline guided search heuristic.

After symbolic execution completes, ECLIPSE generates output with a human-readable summary of discovered vulnerabilities. This summary includes the type of memory safety violation and concrete input values that trigger the bug. There are inherent risks that including this step would introduce, including if an attacker is able to run ECLIPSE on a program to generate exploits automatically before the program developer is able to respond. However, automatic exploit generation is valuable for testing and demonstration purposes.

VI. EVALUATION

To evaluate the effectiveness of ECLIPSE, we aim to explore the following: (1) correctness finding intentionally-added memory safety vulnerabilities in mock programs, (2) real-world effectiveness at finding memory safety vulnerabilities in existing programs, and (3) performance improvements over baseline symbolic execution.

First, as a baseline, we will create mock utility programs that intentionally include different types of memory safety violations. We will then run ECLIPSE on these programs to ensure that it can correctly identify the violations we

introduced. To show the results of this experiment, we will present a table containing the name of each mock utility program and a description of the memory safety violations present in it. We will mark each detected violation with a checkmark (✓). The goal of this initial experiment is to ensure that all injected violations are detected. If any violations are missed, we would continue to iterate and fix issues with ECLIPSE until all the violations we introduced are found.

Next, we will evaluate ECLIPSE’s effectiveness in successfully finding memory safety violations in existing, reasonably sized C-based Linux CLI tools. We will present our results in a table where one column contains the program name along with a brief description of its functionality, and a second column lists the number of detected violations along with a short description of each. While we acknowledge that many real-world CLI tools may be well-maintained and contain few or no vulnerabilities, our ultimate goal is to show ECLIPSE’s ability to discover real bugs in existing software and highlight its real-world effectiveness.

Finally, we will evaluate the effectiveness of ECLIPSE’s pre-processing optimizations in mitigating path explosion and improving the speed of symbolic execution compared to unmodified KLEE. To do this, we perform an experiment where we progressively add each pre-processor-introduced optimization and measure its impact on symbolic execution time. We begin by measuring execution time using unmodified KLEE. We then incrementally add loop bounding, GPIO modeling, structured input modeling, and finally guided search, recording execution time after each addition. We will present these results in a table for each program, allowing us to compare the impact of each optimization and determine which techniques contribute most significantly to reducing path explosion and improving performance.

VII. PLAN OF WORK

The minimum goal of our research (and what we completed by the April 8th check-in) is to:

- 1) Create a minimally viable version of ECLIPSE that accepts an *already-preprocessed* C command line utility program, compiles the program to LLVM bytecode using Clang, and generates concrete inputs that trigger memory safety violations in the program using symbolic execution (KLEE). KLEE should also utilize an SMT solver (STP) to help mitigate path explosion.
- 2) Create a few example *pre-processed* utility programs, written in C, to test ECLIPSE on. These programs will include intentionally placed bugs to make them memory-unsafe.
- 3) Show that ECLIPSE can successfully identify the intentionally placed bugs in the input programs.

The expected goal of our research is to:

- 1) Build the pre-processor for ECLIPSE that adds the core optimizations mentioned in the paper, including adding loop bounds and modeling GPIO inputs + structured

input with constrained symbolic variables. The pre-processor should also identify “risky” functions for guided symbolic execution.

- 2) Add the pre-processor to the ECLIPSE workflow and develop the guided search strategy for KLEE.
- 3) Create a few example *unprocessed* utility programs, written in C, to test ECLIPSE on. These programs will include intentionally placed bugs to make them memory-unsafe.
- 4) Show that ECLIPSE can successfully identify the intentionally placed bugs in the input programs.
- 5) Test our tool on existing C-based Linux CLI tools to find memory violations.

The stretch goals of our research is to:

- 1) Improve the way vulnerabilities found in the program are reported, perhaps with better formatting or even an LLM-generated summary of the vulnerability for the developer to better understand it.
- 2) Improve our guided search strategy to be more robust.

VIII. ATTRIBUTIONS

All three members of our group compiled a collection of related papers that informed the design of our approach and potential pitfalls we would need to protect against. We also discussed the goals for our project and the timelines that we want to aspire to uphold. Using these papers as references, Will and Ben completed the first draft of the state-of-the-art and background sections. Ben also worked on the justification section. Ajay architected the approach for ECLIPSE, created diagrams, and wrote the introduction section and sections describing the threat model, approach, and evaluation. The team worked together on the proposed plan section of the paper. All three members together iterated on the final paper and contributed to the writing of various sections to ensure a coherent understanding of our work, goals, and how existing literature informs our approach.

IX. AI USE

Our team used AI tools, including Anthropic’s Claude and OpenAI’s ChatGPT, to aid us in developing our proposal. We used AI to help us iterate on project ideas and determine whether our idea would be feasible given the amount of allotted time to complete the project. We utilized AI for brainstorming, which helped us develop potential approaches and identify key related literature and tools. This also allowed us to establish a basis for our development strategy and to assess our methods against potential issues. Despite using AI for ideation and brainstorming, aside from helping locate grammar issues, we did *not* use AI to generate the written components of this project proposal. We also came up with the name of our tool, ECLIPSE, ourselves.

REFERENCES

- [1] C. Cadar, D. Dunbar, and D. Engler, 'KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs', in Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation, San Diego, California, 2008, pp. 209–224.
- [2] R. Baldoni, E. Coppa, D. C. D'Elia, C. Demetrescu, and I. Finocchi, 'A Survey of Symbolic Execution Techniques,' *ACM Comput. Surv.*, vol. 51, no. 3, Art. 50, May 2018, doi: 10.1145/3182657.
- [3] J. C. King, 'Symbolic execution and program testing', *Commun. ACM*, vol. 19, no. 7, pp. 385–394, July 1976.
- [4] C. Cadar, V. Ganesh, P. M. Pawlowski, D. L. Dill, and D. R. Engler, 'EXE: Automatically Generating Inputs of Death', *ACM Trans. Inf. Syst. Secur.*, vol. 12, no. 2, Dec. 2008.
- [5] D. D. Chen, M. Woo, D. Brumley, and M. Egele, 'Towards Automated Dynamic Analysis for Linux-based Embedded Firmware', in Network and Distributed System Security Symposium, 2016.
- [6] D. Davidson, B. Moench, T. Ristenpart, and S. Jha, 'FIE on Firmware: Finding Vulnerabilities in Embedded Systems Using Symbolic Execution', in 22nd USENIX Security Symposium (USENIX Security 13), 2013, pp. 463–478.
- [7] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, 'Address-Sanitizer: A Fast Address Sanity Checker', in 2012 USENIX Annual Technical Conference (USENIX ATC 12), 2012, pp. 309–318.
- [8] G. Lacombe, D. Feliot, E. Boespflug, and M.-L. Potet, 'Combining static analysis and dynamic symbolic execution in a toolchain to detect fault injection vulnerabilities', *Journal of Cryptographic Engineering*, vol. 14, no. 1, pp. 147–164, Jan. 2023.
- [9] T. Jung, F. Ritter, and S. Hack, 'Memory Safety Instrumentations in Practice: Usability, Performance, and Security Guarantees', in Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization, Las Vegas, NV, USA, 2025, pp. 390–404.
- [10] J. Vijiutik, L. Perkov, and A. Krog, 'Bug detection in embedded environments by fuzzing and symbolic execution', in 2020 43rd International Convention on Information, Communication and Electronic Technology (MIPRO), 2020, pp. 1218–1223.
- [11] K. J. Zaddach, L. Bruno, A. Francillon, and D. Balzarotti, 'AVATAR: A Framework to Support Dynamic Security Analysis of Embedded Systems' Firmwares', in Network and Distributed System Security Symposium, 2014.
- [12] L. A. Clarke, 'A program testing system', in Proceedings of the 1976 Annual Conference, Houston, Texas, USA, 1976, pp. 488–491.

ECLIPSE: Verifying Memory Safety of CLI Programs for Embedded Linux Systems

Ajay Gandecha
UNC-Chapel Hill

Ben Chesser
UNC-Chapel Hill

Will Zahrt
UNC-Chapel Hill

Abstract—Embedded systems can be found everywhere in our 21st century lives. The consequences of an attacker exploiting memory-safety vulnerabilities in software running on these systems are catastrophic, allowing attackers to read sensors or interact with the real world through these computers. To address this problem, we introduce a tool, ECLIPSE, for formally verifying C-based command-line utility programs running on embedded Linux-based systems, like the Raspberry Pi, for memory safety violations. ECLIPSE will rely on symbolic execution to systematically explore all feasible program paths and generate concrete inputs to the program that trigger memory safety violations. To help limit path explosion, our tool will utilize an SMT solver that can identify and prune infeasible paths early, and pre-processing that adds bounds on loop iterations and inputs as needed. In order to prove success, we plan to create utility programs that intentionally include memory safety violations and run our tool on it to show exploits. As a stretch goal, our tool may also include an automatic exploit generator, which uses the results of symbolic execution to generate exploits that automatically trigger memory violations in the input C command line program.

I. INTRODUCTION

Though often overlooked, embedded systems support important infrastructure in our everyday lives. Fields including electronics, healthcare, emergency systems, home appliances, locomotion, and automation all rely on these embedded systems functioning safely and as designed.

Embedded computers, including the Raspberry Pi, include general-purpose input / output (GPIO) pins, which serve as the interface between the processor and external hardware components. These pins are software-controlled. Through GPIO pins, software can read both inputs (for example, from sensors or switches) and send outputs (for example, to power lights or motors). If an attacker could exploit software programs running on such embedded systems, they could gain unauthorized access to the hardware itself, including these GPIO pins. This could be especially catastrophic, since the attacker could potentially gain access to observe or interact with the physical world via the affected computer.

Prior work on symbolic execution for embedded systems has largely focused on real firmware targeting specific microcontrollers [6] or full firmware image analysis frameworks requiring hardware emulation [11]. Tools like KLEE have been applied extensively to general-purpose software but not specifically to the class of command-line utilities that run on embedded Linux systems like the Raspberry Pi. The goal of this paper is to formally verify C-based command-line utility

programs running on embedded Linux-based systems, such as the Raspberry Pi, for memory safety violations.

To accomplish this goal, we will introduce ECLIPSE (Embedded CLI Program Symbolic Executor), a tool that finds memory safety vulnerabilities in provided C-based command line utility programs. ECLIPSE relies on symbolic execution to systematically explore all feasible program paths and generate concrete inputs to the program that trigger memory safety violations. Symbolic execution works well with our threat model of a malicious attacker who can control inputs but not modify the program directly [1]. ECLIPSE utilizes KLEE, a leading symbolic execution engine with demonstrated success in detecting memory safety violations in C programs [11] [8].

The primary challenge of symbolic execution is path explosion [2]. To help mitigate this, KLEE leverages an SMT solver to remove infeasible paths during execution. However, this alone is insufficient for CLI-based embedded programs, which often exhibit characteristics that cause the symbolic execution engine to explore a large number of unrealistic, semantically invalid paths, worsening path explosion.

First, CLI programs often accept structured inputs, such as flags and options, whose values are typically constrained (e.g., enumerations or bounded numeric ranges and string lengths). Second, embedded programs often interact with hardware, for example by reading data from GPIO pins, whose values are inherently constrained to a low signal (0) or a high signal (1). However, standard symbolic execution treats both structured inputs and GPIO data as unconstrained, leading to the exploration of semantically invalid paths and contributing to path explosion.

To address this, we introduce a dedicated, automated pre-processor that algorithmically transforms input CLI programs into analysis-friendly representations that limit unnecessary path exploration. The pre-processor analyzes the input program, identifies structured inputs and GPIO reads, and models them as symbolic variables constrained to their valid domains. It additionally generates harness code to enforce these constraints during symbolic execution and applies bounds to loop iterations, which are a common source of path explosion due to unbounded branching [2].

Finally, the pre-processor performs static analysis to identify functions containing code patterns associated with memory-safety risks, such as symbolic array indexing, pointer arithmetic, and calling memory-mutating functions. Using this information, the pre-processor guides symbolic execution toward

these high-risk functions to improve bug-finding efficiency while de-emphasizing the exploration of low-risk paths.

We unify this automated pre-processor with KLEE to create our tool, ECLIPSE, which accepts an input C program and identifies memory safety violations within the program.

To test its effectiveness, we first create mock utility programs that intentionally include memory safety violations. Then, we run ECLIPSE on these programs to ensure that it can correctly identify all the memory safety violations that we introduced. We then run ECLIPSE on larger, existing C-based Linux CLI tools to determine if it can discover any new, previously undiscovered bugs. We acknowledge the possibility that we may correctly implement our entire design for ECLIPSE and still not detect vulnerabilities in open-source programs, especially if they are well-maintained.

We plan to spend the rest of the semester completing this project. We anticipate no additional monetary cost to us, since we already have possession of a Raspberry Pi system if needed for testing purposes.

II. STATE OF THE ART

Foundations of Symbolic Execution

The original papers introducing the symbolic execution method are important background since the goal of our project is to show that this technique is effective in identifying memory safety violations in command-line utilities running on embedded systems [3] [12].

Similarly, a comprehensive survey of symbolic execution techniques gathers and summarizes symbolic execution's strengths, weaknesses, and challenges (like path explosion) through a comprehensive survey [2]. This provides the theoretical framework for the challenges we will face developing ECLIPSE and the solutions we can employ, which include constraint solving via an SMT solver, and constraint reduction.

The KLEE symbolic execution engine is the foundational background for our project, since KLEE is the symbolic execution engine that we plan to integrate into ECLIPSE to verify input programs [1]. KLEE can automatically generate high-coverage test cases by efficiently exploring program paths. In fact, our work is an applied case study of KLEE in the embedded Linux domain.

EXE is a bug-finding tool that creates exploits based on vulnerabilities discovered during symbolic execution [4]. This is relevant to our project because we plan on generating runnable exploits for the command-line programs we are verifying.

Symbolic Execution for Embedded Systems

FIE adapted symbolic execution specifically for microcontroller firmware running on embedded systems [6]. Our work is domain-adjacent because we are extending the application of symbolic execution up the stack from firmware to command-line utilities.

Automated dynamic analysis enables security inspection on embedded systems without physical hardware by emulating the full Linux system environment [5]. Our project is

complementary to this work because, instead of focusing on dynamic analysis through emulation, we are applying symbolic execution to find vulnerabilities.

Avatar combines KLEE's symbolic execution with QEMU's full-system emulation to analyze embedded firmware [11]. Our work is conceptually aligned with this, but instead of focusing on firmware and hardware interaction, we are targeting user-space command line utilities through Linux devices.

Memory Safety Instrumentation and Detection

AddressSanitizer inserts runtime checks around memory addresses, which enables detection of memory errors during program execution [7]. This is important background for our project because we do something similar but instead use symbolic execution to explore execution paths and detect errors before they occur in production.

An analysis of tradeoffs between usability and performance overhead for various compile-time memory safety instrumentations, conducted through the MemInstrument framework, provides context and comparison for our technique, which trades performance for exhaustive path coverage [9].

Hybrid Approaches

Prior approaches to bug detection in embedded environments applies both fuzzing and symbolic execution to detect crashes and vulnerabilities in OpenWRT, which is highly complementary to our project [10]. Instead of casting a wide net over embedded OS distributions, however, we are doing a more targeted project into command-line utilities.

A toolchain combining static analysis and dynamic symbolic execution to detect fault injection vulnerabilities first applies static analysis to identify promising targets and then uses KLEE for targeted symbolic execution [8]. This is directly relevant to our stretch goal of further limiting path explosion through means other than setting bounds.

III. BACKGROUND

Memory safety is a safety property of programs that ensure memory-accesses are well defined and follow the intended rules of the language. Memory-safe programs will never read or write to a memory location in ways that violate language specification or program assumptions. The most common types of memory safety violations include buffer overflows, null pointer de-references, out-of-bounds reads, and memory leaks. These vulnerabilities are particularly prevalent in the C family of programming languages because these languages give direct control to the developer over memory management without built-in safety checks.

Memory safety matters to embedded Linux systems (like the Raspberry Pi) because they rely on small command-line utilities written in C for critical tasks. When a privileged utility contains a memory safety violation, an attacker who controls inputs can potentially do malicious actions such as crashing the system, corrupting data, executing arbitrary code, read sensor inputs or interact with the physical world via GPIO pins, or permanently damaging the hardware.

KLEE is a symbolic execution engine that operates on LLVM bytecode. First, a target C program is compiled to LLVM bytecode using a compiler like Clang. Then, the programmer identifies which inputs should be treated as symbols using KLEE’s intrinsic functions. From there, KLEE executes the bytecode to fork at each symbolic branch and collect path constraints. KLEE performs constraint solving from there by querying the STP constraint solver to check path feasibility and generate concrete values for test cases. Finally, KLEE does error detection and generates files containing concrete input values that allow one to reproduce the path to an error. We plan to use KLEE as the symbolic execution engine that powers ECLIPSE.

IV. THREAT MODEL

The goal of ECLIPSE is to formally verify command line utility programs for embedded Linux devices against memory safety violations. We assume that the attacker can control the inputs to the command line utility, including command line arguments, potential hardware properties including GPIO configurations, and sensor input values. We also assume that the attacker has read-only access to the source code of the program that is the target of attack. The attacker having access to the source code is likely, especially in situations where an attacker attempts to exploit open-source programs.

We also assume that the attacker has no write access to the program’s source code or the program’s binary post-compilation. The goal of our tool is to verify memory safety properties for programs when run with malicious inputs, not tampered code. If the attacker could modify the program directly, they could simply insert malicious behaviour directly into the code rather than needing to exploit a bug to violate a memory safety property.

V. APPROACH

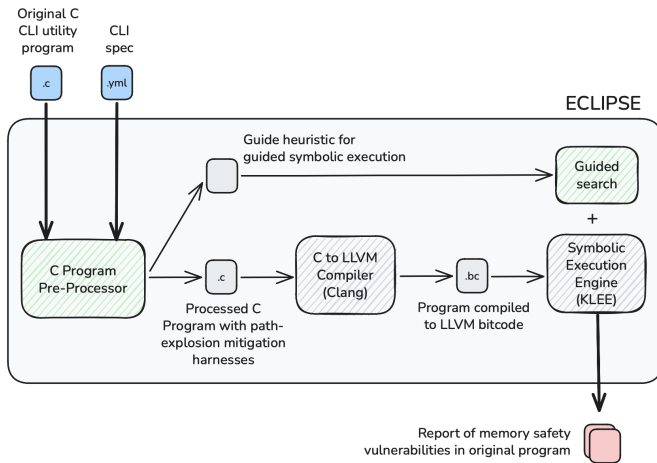


Fig. 1. ECLIPSE’s internal process

At a high level, ECLIPSE operates as a pipeline-style workflow. It accepts two inputs: (1) a C program and (2) a CLI specification file describing the valid input structure.

The program is first transformed by a custom pre-processor to constrain inputs, model hardware interactions, and bound potential sources of path explosion. ECLIPSE then compiles the transformed program into LLVM bytecode using Clang and performs symbolic execution using KLEE with a guided search strategy. Finally, ECLIPSE produces a report describing discovered vulnerabilities along with concrete inputs that reproduce them. An overview of ECLIPSE’s workflow is shown in Figure 1.

ECLIPSE takes in two primary inputs: (1) a C program which is the target CLI utility to analyze, and (2) a YAML-based CLI specification file. The CLI specification file describes the structure of valid input to the target C program and their constraints, such as supported flags and options, data types, and valid values (e.g., enumerations or bounds on integer ranges or string lengths).

For example, consider a program `tool.c` that accepts a `--pin` integer argument that maps to a physical GPIO pin on a Raspberry Pi. These devices have 26 GPIO pins, so `--pin` should only ever accept integer values in the range `{1, 26}`. Given this structured argument, the CLI specification file for this program might look like the following:

```

program: tool
arguments:
- name: pin
  flag: --pin
  type: int
  required: true
  min: 0
  max: 26

```

From there, ECLIPSE begins the pre-processing stage by parsing the input C program into an abstract syntax tree (AST). This intermediate representation enables ECLIPSE to algorithmically analyze the program and perform any necessary transformations during the pre-processing stage. Analyzing an AST is more robust than pattern-matching using regex because it analyzes the structure of code directly and is not affected by syntactic anomalies introduced by the developer.

Once the input program is parsed into an AST, the pre-processor examines the program to replace concrete CLI inputs with symbolic variables constrained by the program’s provided CLI specification. Each argument is mapped to a symbolic variable, and constraints are applied based on its declared type. For example, integer arguments may be bound with specified ranges, enumerated arguments may be restricted to a set of possible values, and string inputs may be constrained by length. The pre-processor then generates a harness that enforces these constraints using KLEE’s symbolic execution intrinsic functions. By modeling inputs this way, ECLIPSE ensures that the symbolic execution engine avoids exploring semantically invalid input combinations, which reduces path explosion without sacrificing coverage.

For example, a sample generated harness for the `tool.c` example above, given its CLI specification, might look like the following:

```
int harness_main() {
    int sym_pin;
    char *argv[3];

    // Creates a symbolic variable
    klee_make_symbolic(
        &sym_pin, sizeof(sym_pin), "pin"
    );

    // Constrains the symbolic variable
    klee_assume(sym_pin >= 1 && sym_pin <= 26);

    argv[0] = "tool";
    argv[1] = "--pin";
    argv[2] = int_to_string(sym_pin);

    return main(3, argv);
}
```

With this harness in place, KLEE will never explore paths in the target program where the pin's value is outside its semantically meaningful range of $\{1, 26\}$.

The pre-processor then examines the program to identify GPIO read operations in the AST and replaces them with symbolic variables constrained to the valid domain $\{0, 1\}$. The pre-processor generates a harness that enforces this constraint. Embedded CLI programs frequently interact with hardware through GPIO pins whose inputs are inherently constrained to a low signal (0) or a high signal (1), but by default, the symbolic execution engine would treat these as unconstrained values. Replacing GPIO read operations in the program with constrained symbolic variables ensures that symbolic execution reflects realistic hardware behavior and avoids exploring semantically invalid hardware states, which also helps to reduce path explosion.

Next, the pre-processor analyzes the AST to identify loops and modifies the program to manually add bounds on the maximum number of iterations to limit path explosion. The maximum number of iterations can be customized, but by default, bounds are selected based on the loop's structure (e.g., based on a variable input or fixed with a constant). For example, given an unconstrained loop:

```
while(condition) {
    foo(); // Run some process
}
```

The pre-processor would bound this loop like so:

```
int n = 0;
while(condition && n < MAX_ITERATIONS) {
    foo(); // Run some process
    n += 1;
}
```

This way, no matter when `condition` changes to false, the loop always iterates at most `MAX_ITERATIONS` times before terminating.

Finally, the pre-processor performs static analysis by analyzing the AST to identify functions containing code patterns associated with memory-safety risks, such as symbolic array indexing, pointer arithmetic, and calling memory-mutating functions. Functions containing these are then recorded in a *risk file* that ranks functions based on their likelihood of containing memory safety vulnerabilities. This information is later used to guide the symbolic execution engine towards high-risk code with the goal of improving bug-finding efficiency. This risk file, along with the final processed C code file, are output by the pre-processor.

Once pre-processing is finished, ECLIPSE then compiles the processed program. KLEE accepts programs in the format of LLVM bitcode. GCC, the traditional C compiler, compiles C programs into native binary, which is ISA-specific and dependent on the target hardware. Instead of using GCC, ECLIPSE compiles the processed program with Clang, a compiler that compiles C code directly into LLVM bitcode.

Finally, ECLIPSE uses KLEE to perform symbolic execution on the compiled program. KLEE also uses STP, an SMT solver that prunes infeasible paths and generates concrete inputs that satisfy the path constraints generated during symbolic execution. To improve bug-finding efficiency, ECLIPSE augments KLEE's default search strategy with a custom guided search heuristic. This heuristic prioritizes execution paths that lead toward the risky functions noted in the risk file created by the pre-processor. States associated with higher-risk functions are selected more frequently. By default, KLEE interleaves the pre-built `random-path` and `nurs:covnew` heuristics as its search strategy. ECLIPSE replaces the `random-path` with its custom guided heuristic. This approach is intentionally lightweight to ensure compatibility with KLEE's existing architecture and so it is reasonably scoped given our time constraint. As a stretch goal, we may try to develop strategies to improve our baseline guided search heuristic.

After symbolic execution completes, ECLIPSE generates output with a human-readable summary of discovered vulnerabilities. This summary includes the type of memory safety violation and concrete input values that trigger the bug. There are inherent risks that including this step would introduce, including if an attacker is able to run ECLIPSE on a program to generate exploits automatically before the program developer is able to respond. However, automatic exploit generation is valuable for testing and demonstration purposes.

VI. EVALUATION

To evaluate the effectiveness of ECLIPSE, we aim to explore the following: (1) correctness finding intentionally-added memory safety vulnerabilities in mock programs, (2) real-world effectiveness at finding memory safety vulnerabilities in existing programs, and (3) performance improvements over baseline symbolic execution.

First, as a baseline, we will create mock utility programs that intentionally include different types of memory safety violations. We will then run ECLIPSE on these programs to ensure that it can correctly identify the violations we

introduced. To show the results of this experiment, we will present a table containing the name of each mock utility program and a description of the memory safety violations present in it. We will mark each detected violation with a checkmark (✓). The goal of this initial experiment is to ensure that all injected violations are detected. If any violations are missed, we would continue to iterate and fix issues with ECLIPSE until all the violations we introduced are found.

Next, we will evaluate ECLIPSE’s effectiveness in successfully finding memory safety violations in existing, reasonably sized C-based Linux CLI tools. We will present our results in a table where one column contains the program name along with a brief description of its functionality, and a second column lists the number of detected violations along with a short description of each. While we acknowledge that many real-world CLI tools may be well-maintained and contain few or no vulnerabilities, our ultimate goal is to show ECLIPSE’s ability to discover real bugs in existing software and highlight its real-world effectiveness.

Finally, we will evaluate the effectiveness of ECLIPSE’s pre-processing optimizations in mitigating path explosion and improving the speed of symbolic execution compared to unmodified KLEE. To do this, we perform an experiment where we progressively add each pre-processor-introduced optimization and measure its impact on symbolic execution time. We begin by measuring execution time using unmodified KLEE. We then incrementally add loop bounding, GPIO modeling, structured input modeling, and finally guided search, recording execution time after each addition. We will present these results in a table for each program, allowing us to compare the impact of each optimization and determine which techniques contribute most significantly to reducing path explosion and improving performance.

VII. PLAN OF WORK

The minimum goal of our research (and what we completed by the April 8th check-in) is to:

- 1) Create a minimally viable version of ECLIPSE that accepts an *already-preprocessed* C command line utility program, compiles the program to LLVM bytecode using Clang, and generates concrete inputs that trigger memory safety violations in the program using symbolic execution (KLEE). KLEE should also utilize an SMT solver (STP) to help mitigate path explosion.
- 2) Create a few example *pre-processed* utility programs, written in C, to test ECLIPSE on. These programs will include intentionally placed bugs to make them memory-unsafe.
- 3) Show that ECLIPSE can successfully identify the intentionally placed bugs in the input programs.

The expected goal of our research is to:

- 1) Build the pre-processor for ECLIPSE that adds the core optimizations mentioned in the paper, including adding loop bounds and modeling GPIO inputs + structured

input with constrained symbolic variables. The pre-processor should also identify “risky” functions for guided symbolic execution.

- 2) Add the pre-processor to the ECLIPSE workflow and develop the guided search strategy for KLEE.
- 3) Create a few example *unprocessed* utility programs, written in C, to test ECLIPSE on. These programs will include intentionally placed bugs to make them memory-unsafe.
- 4) Show that ECLIPSE can successfully identify the intentionally placed bugs in the input programs.
- 5) Test our tool on existing C-based Linux CLI tools to find memory violations.

The stretch goals of our research is to:

- 1) Improve the way vulnerabilities found in the program are reported, perhaps with better formatting or even an LLM-generated summary of the vulnerability for the developer to better understand it.
- 2) Improve our guided search strategy to be more robust.

VIII. ATTRIBUTIONS

All three members of our group compiled a collection of related papers that informed the design of our approach and potential pitfalls we would need to protect against. We also discussed the goals for our project and the timelines that we want to aspire to uphold. Using these papers as references, Will and Ben completed the first draft of the state-of-the-art and background sections. Ben also worked on the justification section. Ajay architected the approach for ECLIPSE, created diagrams, and wrote the introduction section and sections describing the threat model, approach, and evaluation. The team worked together on the proposed plan section of the paper. All three members together iterated on the final paper and contributed to the writing of various sections to ensure a coherent understanding of our work, goals, and how existing literature informs our approach.

IX. AI USE

Our team used AI tools, including Anthropic’s Claude and OpenAI’s ChatGPT, to aid us in developing our proposal. We used AI to help us iterate on project ideas and determine whether our idea would be feasible given the amount of allotted time to complete the project. We utilized AI for brainstorming, which helped us develop potential approaches and identify key related literature and tools. This also allowed us to establish a basis for our development strategy and to assess our methods against potential issues. Despite using AI for ideation and brainstorming, aside from helping locate grammar issues, we did *not* use AI to generate the written components of this project proposal. We also came up with the name of our tool, ECLIPSE, ourselves.

REFERENCES

- [1] C. Cadar, D. Dunbar, and D. Engler, 'KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs', in *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, San Diego, California, 2008, pp. 209–224.
- [2] R. Baldoni, E. Coppa, D. C. D'Elia, C. Demetrescu, and I. Finocchi, 'A Survey of Symbolic Execution Techniques,' *ACM Comput. Surv.*, vol. 51, no. 3, Art. 50, May 2018, doi: 10.1145/3182657.
- [3] J. C. King, 'Symbolic execution and program testing', *Commun. ACM*, vol. 19, no. 7, pp. 385–394, July 1976.
- [4] C. Cadar, V. Ganesh, P. M. Pawlowski, D. L. Dill, and D. R. Engler, 'EXE: Automatically Generating Inputs of Death', *ACM Trans. Inf. Syst. Secur.*, vol. 12, no. 2, Dec. 2008.
- [5] D. D. Chen, M. Woo, D. Brumley, and M. Egele, 'Towards Automated Dynamic Analysis for Linux-based Embedded Firmware', in *Network and Distributed System Security Symposium*, 2016.
- [6] D. Davidson, B. Moench, T. Ristenpart, and S. Jha, 'FIE on Firmware: Finding Vulnerabilities in Embedded Systems Using Symbolic Execution', in *22nd USENIX Security Symposium (USENIX Security 13)*, 2013, pp. 463–478.
- [7] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, 'Address-Sanitizer: A Fast Address Sanity Checker', in *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, 2012, pp. 309–318.
- [8] G. Lacombe, D. Feliot, E. Boespflug, and M.-L. Potet, 'Combining static analysis and dynamic symbolic execution in a toolchain to detect fault injection vulnerabilities', *Journal of Cryptographic Engineering*, vol. 14, no. 1, pp. 147–164, Jan. 2023.
- [9] T. Jung, F. Ritter, and S. Hack, 'Memory Safety Instrumentations in Practice: Usability, Performance, and Security Guarantees', in *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, Las Vegas, NV, USA, 2025, pp. 390–404.
- [10] J. Vijiutik, L. Perkov, and A. Krog, 'Bug detection in embedded environments by fuzzing and symbolic execution', in *2020 43rd International Convention on Information, Communication and Electronic Technology (MIPRO)*, 2020, pp. 1218–1223.
- [11] K. J. Zaddach, L. Bruno, A. Francillon, and D. Balzarotti, 'AVATAR: A Framework to Support Dynamic Security Analysis of Embedded Systems' Firmwares', in *Network and Distributed System Security Symposium*, 2014.
- [12] L. A. Clarke, 'A program testing system', in *Proceedings of the 1976 Annual Conference*, Houston, Texas, USA, 1976, pp. 488–491.